COURSE OUTLINE

Gennoor Tech
TRAIN. INNOVATE. BUILD.

Cisco ACI Upgrade Automation with Claude

Plan, validate and execute ACI fabric upgrades the Cisco way, with Claude drafting and code executing

3 days (8 hours per day) | Intermediate to advanced | Instructor-led, hands-on labs | Capstone project

About this course

Who should attend

Enterprise network operations, data centre, NOC and network automation engineers who run Cisco ACI fabrics, in India and worldwide.

Prerequisites

- Working knowledge of Cisco ACI: APIC, leaf and spine, tenants, vPC
- Basic Python and REST API calls with JSON
- Familiarity with change management and maintenance windows
- Familiarity with an AI coding assistant is helpful, not required

What you will be able to do

- Describe the Cisco-recommended ACI upgrade sequence: release selection, pre-upgrade validation, APIC cluster first, then switch groups, then post-checks
- Build a Python client for the APIC REST API that discovers the fabric, runs the pre-upgrade validator and returns typed JSON
- Use Claude Code, the Claude Agent SDK and an MCP server to plan upgrade groups and maintenance windows as schema-validated structured output
- Create firmware and maintenance policies with deterministic Python and Ansible only after a human approves the plan in a GitHub pull request and a ServiceNow change
- Produce pre and post upgrade evidence, a comparison report, a pause or rollback decision and a complete audit trail

Tools and technologies

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

Cisco ACI / APIC REST API | Claude Code | Claude Agent SDK | Python | Pydantic | Ansible cisco.aci collection | ServiceNow Table API | GitHub pull requests | GitHub Actions | pytest | JSON Schema

Table of contents

DAY 1 4 modules

MODULE 1

The Cisco ACI upgrade process and where upgrades go wrong

- Cisco's recommended sequence: target release and release notes, compatibility matrix, pre-upgrade validation, APIC cluster, switch upgrade groups, post-checks
- Why vPC peers never share a group, why the cluster must be fully fit, and what bootflash space, faults and NTP drift do to an upgrade
- Firmware repository, maintenance policies, maintenance groups and the scheduler as APIC objects
- The teaching pattern: Claude plans and explains, deterministic code executes, a human approves, everything is logged

HANDS-ON LAB
Walk the lab fabric in the APIC GUI and API Inspector, map each Cisco process step to the APIC objects it touches, and write the workflow brief.

DELIVERABLE
ACI upgrade workflow brief.

MODULE 2

Claude Code, Agent SDK and MCP setup for the ACI workflow

- Installing Claude Code and the Claude Agent SDK in the project repository with a CLAUDE.md that encodes the upgrade rules
- Building an MCP server that exposes read-only APIC queries (nodes, firmware, faults, health) as tools
- Tool use with structured JSON outputs: defining the inventory and plan schemas Claude must return
- Separating read tools from write actions so the agent can observe but never trigger

HANDS-ON LAB
Stand up the MCP server against the lab APIC and have Claude Code answer fabric questions using only the exposed tools.

DELIVERABLE
Working Claude Code and MCP environment.

Genroor Tech INNOVATE BUILD for more info - https://genroor.com

MODULE 3

Fabric discovery through the APIC REST API

- POST to aaaLogin, token refresh via aaaRefresh, and class and filter queries against fabricNode, topSystem, infraWiNode and firmwareCtrlrRunning
- Typed inventory record: node id, role, model, running version, vPC peer, health score, cluster state
- Pydantic validation, unit tests with pytest, and rejected-row reporting for incomplete nodes

HANDS-ON LAB
Implement the discovery client and tests with Claude Code, inspect every diff, and produce inventory.json for the lab fabric.

DELIVERABLE
Fabric inventory client.

MODULE 4

Pre-upgrade validation and explained failures

- Running the APIC pre-upgrade validator and the Cisco aci-preupgrade-validation-script and parsing their results
- Checking faults, bootflash space, cluster health, NTP, CIMC and configuration backups against the target release
- Claude explains each failed check in plain language, cites the query that produced it and marks unknowns

HANDS-ON LAB
Fill a leaf's bootflash on purpose on the lab fabric, run validation, and have Claude explain the failure and the Cisco-recommended remediation.

DELIVERABLE
Validation report with explained findings.

DAY 2 3 modules

Genoor Tech INNOVATE. BUILD.
For more info - https://genoor.com

MODULE 5

Upgrade planning with structured outputs

- Claude proposes upgrade groups from the inventory: APIC cluster first, vPC peers split, spines balanced, leaves balanced
- Mapping groups to maintenance windows and scheduler times in the fabric's local time zone
- JSON Schema validation of the plan and automatic rejection of plans that break a rule
- Testing the planner with awkward fabrics: odd vPC pairs, mixed models, a degraded APIC

HANDS-ON LAB
Generate the plan for the lab fabric, feed it a fabric with an unpaired vPC leaf, and prove the schema gate rejects the bad plan.

DELIVERABLE
Schema-validated upgrade plan.

MODULE 6

Firmware staging, backups and the change request

- Uploading firmware to the APIC repository with firmwareFirmware and firmwareOSource and verifying checksums
- Configuration export and snapshot with configExportP and configSnapshot before any change
- Creating the ServiceNow change request through the Table API with the plan, windows and risk text drafted by Claude
- Tracking approval state with Flow Designer and blocking execution until the change is approved

HANDS-ON LAB
Stage the target image, take the snapshot, and raise the lab ServiceNow change from the plan with one command.

DELIVERABLE
Staged firmware, snapshot and change request.

Genmoor Technologies
For more info - https://genmoor.com

MODULE 7

Guardrails, approvals and audit for fabric upgrades

- The agent reads; code writes. Maintenance and firmware policies are created only by deterministic Python or Ansible cisco.aci modules
- Plan review and approval in a GitHub pull request with GitHub Actions running the validator and schema checks
- Window enforcement: no trigger outside the approved schedule, no next group while the last one is unhealthy
- Audit log of prompts, tool calls, API calls, approvals and outcomes in a format the change board can read

HANDS-ON LAB

Open the plan as a pull request, try to apply it before approval and watch the gate refuse, then approve and apply to the lab.

DELIVERABLE

Approval gate and audit log.

DAY 3 3 modules

MODULE 8

Executing the APIC cluster and switch group upgrades

- Upgrading the APIC cluster first and polling infraWiNode and firmwareCtrlrFwStatusCont until the cluster is fully fit
- Creating maintenance policies and groups per the plan and triggering each group only inside its window
- Polling firmwareRunning and maintUpgJob per node, with timeouts, retries and a pause when a node does not return
- Claude drafts the status update for the on-call engineer when the runner pauses

HANDS-ON LAB

Run the APIC cluster upgrade and group A on the physical lab fabric, then replay a recorded node-not-returning response from a mocked APIC and watch the runner pause.

DELIVERABLE

Deterministic upgrade runner.

For more info - https://gennoor.com

MODULE 9

Post-upgrade verification, reporting and rollback decisions

- Pre and post snapshots of versions, faults, endpoint counts, routes, vPC and interface state compared by code
- Claude turns the diff into a readable report with a go, hold or roll back recommendation
- Rollback and downgrade options as Cisco documents them, and when to stop the next group
- Lessons captured into the runbook for the next fabric

HANDS-ON LAB
Compare pre and post state for group A, generate the report, and make the go or hold call for group B as a team.

DELIVERABLE
Post-upgrade report and runbook.

MODULE 10

Capstone: end-to-end ACI fabric upgrade with evidence bundle

- Inventory in, upgraded fabric out: discovery, validation, plan, change, approval, execution, verification
- Evidence bundle with validator output, plan, approvals, API log and post-check report
- Demo and limitations statement

HANDS-ON LAB
Capstone build and demo.

DELIVERABLE
End-to-end upgrade workflow and evidence bundle.

Capstone

Capstone project

A repeatable workflow that discovers a Cisco ACI fabric, runs the pre-upgrade validator, has Claude propose a schema-validated upgrade plan, raises and tracks the ServiceNow change, executes the APIC cluster and switch groups through deterministic code only after human approval, and ships a post-upgrade report with a full audit trail.

Assessment

Assessment area	Weight	Evidence

Working functionality	**25%**	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	**20%**	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	**20%**	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	**15%**	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	**10%**	Traceable findings, sources, assumptions, logs
Documentation and presentation	**10%**	Architecture diagram, README, runbook, demo, lessons learned

Engineering principles used throughout

- Model output is a proposal, not a fact: every finding and generated change is validated by deterministic checks and tests.
- Authorization lives outside the model: tool access is enforced by application code, scoped credentials and policy.
- Human approval for consequential actions: deploying a mitigation, changing production configuration, rotating credentials.
- Decisions, evidence, model and tool versions, approvals and outcomes are logged without recording secrets.
- A model adapter and a tool adapter layer keep every build provider-agnostic and enterprise-portable.
- Quality is measured: false positives, false negatives, reliability, latency, cost and safe failure, not just a demo.

© Gennoor Tech · Train. Innovate. Build.